Sohaa Jamadar| 1252090024 | 24 | SEDA (B1)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.datasets import make_classification
import seaborn as sns
```

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report

# Sample cosmetics shop dataset
data = {
    'Age': [22,45,30,35,50,23,40,29,33,48],
    'Salary': [25000,50000,32000,40000,60000,27000,52000,31000,38000,58000],
    'Discount': [10,5,15,10,5,20,5,15,10,5],
    'Purchased': [1,0,1,1,0,1,0,1,1,0]
}

df = pd.DataFrame(data)

# Single predictor
X = df[['Age']]
y = df['Purchased']

X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.3,random_state=42)

model = LogisticRegression()
model.fit(X_train,y_train)

y_pred = model.predict(X_test)

print("Confusion Matrix:\n", confusion_matrix(y_test,y_pred))
print("\nClassification Report:\n", classification_report(y_test,y_pred))
```

```
Confusion Matrix:
 [[1 0]
 [0 2]]

Classification Report:
               precision    recall  f1-score   support

           0       1.00      1.00      1.00         1
           1       1.00      1.00      1.00         2

    accuracy                           1.00         3
   macro avg       1.00      1.00      1.00         3
weighted avg       1.00      1.00      1.00         3
```

Multiple predictors

```
X = df[['Age','Salary','Discount']]
y = df['Purchased']

X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.3,random_state=42)

model_multi = LogisticRegression()
model_multi.fit(X_train,y_train)

y_pred_multi = model_multi.predict(X_test)

print("Confusion Matrix:\n", confusion_matrix(y_test,y_pred_multi))
print("\nClassification Report:\n", classification_report(y_test,y_pred_multi))

# Coefficients
coefficients = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': model_multi.coef_[0]
})
```

```
print("\nModel Coefficients:\n", coefficients)
```

```
Confusion Matrix:
 [[1 0]
 [0 2]]

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         1
           1       1.00      1.00      1.00         2

    accuracy                           1.00         3
   macro avg       1.00      1.00      1.00         3
weighted avg       1.00      1.00      1.00         3


Model Coefficients:
     Feature  Coefficient
0       Age    -0.000019
1    Salary    -0.002542
2  Discount     0.000004
```

```
df = pd.DataFrame(data)
df.rename(columns={'Salary': 'Income', 'Discount': 'Spending_Score', 'Purchased': 'Product_Category'}, inplace=True)

print(df.head())
```

```
   Age  Income  Spending_Score  Product_Category
0   22   25000              10                 1
1   45   50000               5                 0
2   30   32000              15                 1
3   35   40000              10                 1
4   50   60000               5                 0
```

```
X = df[["Age", "Income", "Spending_Score"]]
y = df["Product_Category"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)
```

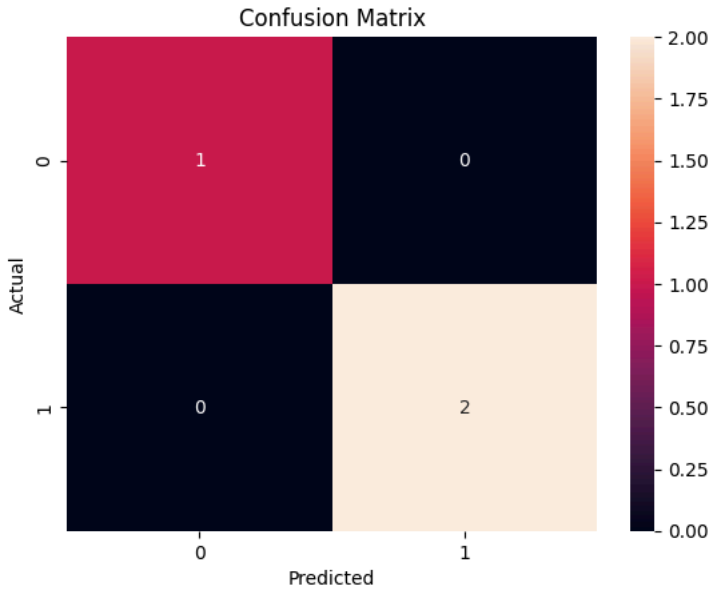
```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
model = LogisticRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

```
print("Confusion Matrix:")
cm = confusion_matrix(y_test, y_pred)
print(cm)

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

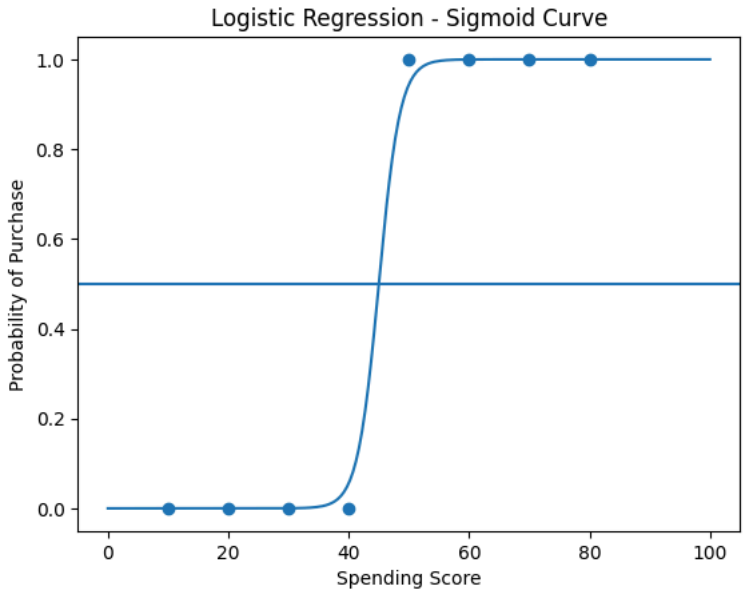
```
Confusion Matrix:
[[1 0]
 [0 2]]

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         1
           1       1.00      1.00      1.00         2

    accuracy                           1.00         3
   macro avg       1.00      1.00      1.00         3
weighted avg       1.00      1.00      1.00         3
```

```
plt.figure()
sns.heatmap(cm, annot=True, fmt='d')
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



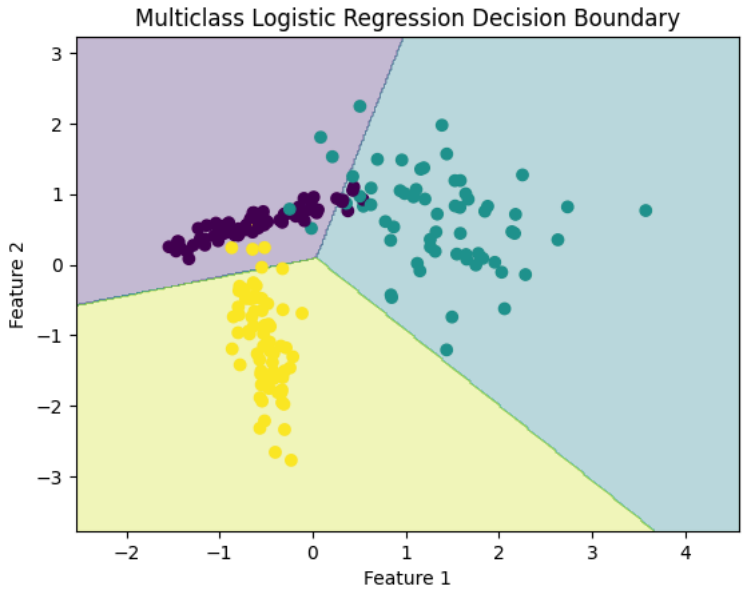
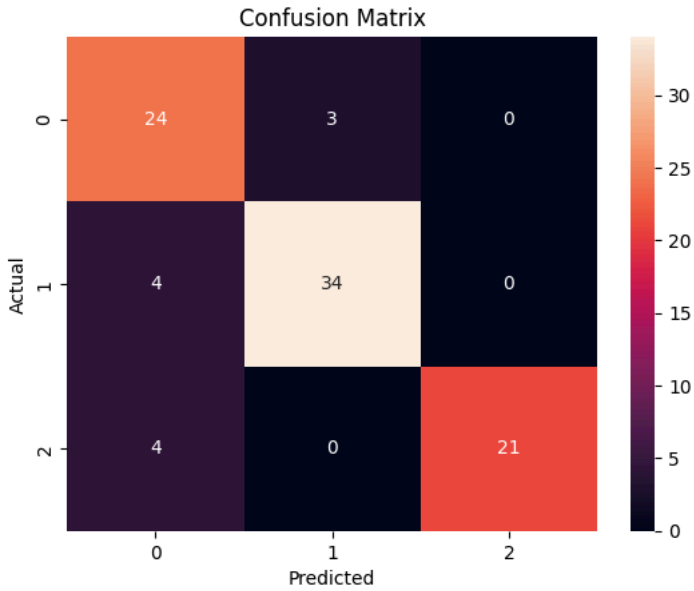

```
#refered not understood
```

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression

# -------------------------------
# 1. Create Simple Dataset
# -------------------------------
# X = Spending Score
# y = Purchase (0 = No, 1 = Yes)

X = np.array([10, 20, 30, 40, 50, 60, 70, 80]).reshape(-1, 1)
y = np.array([0, 0, 0, 0, 1, 1, 1, 1])

# -------------------------------
# 2. Train Logistic Regression
# -------------------------------
model = LogisticRegression()
model.fit(X, y)

# -------------------------------
# 3. Generate Smooth Values for Curve
# -------------------------------
X_test = np.linspace(0, 100, 200).reshape(-1, 1)
y_prob = model.predict_proba(X_test)[:, 1]

# -------------------------------
# 4. Plot Sigmoid Curve
# -------------------------------
plt.figure()
plt.scatter(X, y)
plt.plot(X_test, y_prob)
plt.axhline(0.5)
plt.title("Logistic Regression - Sigmoid Curve")
plt.xlabel("Spending Score")
plt.ylabel("Probability of Purchase")
plt.show()

# -------------------------------
# 5. Decision Boundary
# -------------------------------
b0 = model.intercept_[0]
b1 = model.coef_[0][0]
decision_boundary = -b0 / b1

print("Decision Boundary (Spending Score):", decision_boundary)
```




```
Decision Boundary (Spending Score): 44.999774487572886
```

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix
import seaborn as sns

# ---------------------------------------
# 1. Generate Multiclass Dataset (3 Classes)
# ---------------------------------------
X, y = make_classification(n_samples=300,
                           n_features=2,
                           n_informative=2,
                           n_redundant=0,
                           n_classes=3,
                           n_clusters_per_class=1,
                           random_state=42)

# ---------------------------------------
# 2. Train-Test Split
# ---------------------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

# ---------------------------------------
# 3. Feature Scaling
# ---------------------------------------
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# ---------------------------------------
# 4. Train Multiclass Logistic Regression
# ---------------------------------------
model = LogisticRegression(multi_class='ovr')
model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)

# ---------------------------------------
# 5. Plot Decision Boundary
# ---------------------------------------
x_min, x_max = X_train[:, 0].min() - 1, X_train[:, 0].max() + 1
y_min, y_max = X_train[:, 1].min() - 1, X_train[:, 1].max() + 1

xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02),
                     np.arange(y_min, y_max, 0.02))

Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.figure()
plt.contourf(xx, yy, Z, alpha=0.3)
```

```
plt.scatter(X_train[:, 0], X_train[:, 1], c=y_train)
plt.title("Multiclass Logistic Regression Decision Boundary")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()

# ---------------------------------------
# 6. Confusion Matrix Visualization
# ---------------------------------------
cm = confusion_matrix(y_test, y_pred)

plt.figure()
sns.heatmap(cm, annot=True, fmt='d')
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:1256: FutureWarning: 'multi_class' was deprecated
  warnings.warn(
```